

flow-to-apex

User Guide — setup, daily workflow, and migration steps for converting Salesforce Record-Triggered Flows into bulk-safe Apex.

1. Overview

flow-to-apex is a command-line tool that converts a Salesforce Record-Triggered Flow (`.flow-meta.xml`) into a matching Apex Trigger and Handler class. The output is laid out for direct use inside a Salesforce DX project (`force-app/main/default/`) and is designed to be a clean starting point for a developer to finish, test, and deploy.

The tool focuses on one thing well: turning a Flow's logic into Apex that is **bulk-safe by construction**. When the source Flow contains a Loop with DML inside (the most common reason Flows fail at scale), the generated Apex automatically lifts the DML out of the loop into a `Map<Id, SObject>` collector and issues a single bulk DML statement after the loop closes.

2. Prerequisites

You need the following before using the tool:

- **Node.js 18 or later** installed on your machine. If the Salesforce CLI (`sf`) is already working, Node is already present.
- **A Salesforce DX project** on disk (created via `sf project generate` or cloned from a repository).
- **A Salesforce org connected to the project** via `sf org login web`.
- **VS Code with the Salesforce Extension Pack** (recommended) for retrieving and deploying metadata through the UI.

3. Installation

The tool is distributed as a self-contained folder (its only dependency, `fast-xml-parser`, is bundled). No `npm install` step is required.

Steps

- Obtain the `flow-to-apex.zip` archive.
- Unzip it to a stable location on your machine. The example used in this guide places it at `/Users/<you>/Desktop/Bold/A_Projects/flow-to-apex/`, but any path works as long as it does not change later.
- Verify that `src/cli.js` is present inside the unzipped folder.

At this point you can already run the tool by typing a full node command, but the next section sets up a short alias so you only need to type the Flow's API name.

4. Setting up the convert-flow shortcut

To avoid pasting the long absolute paths every time, add a small shell function to your shell configuration. This is a one-time setup; afterwards you call the converter with just the Flow's API name.

Step 1 — open your shell config

On macOS the default shell is Zsh. Open the config file in TextEdit:

```
open -e ~/.zshrc
```

If the file does not exist yet, create it first:

```
touch ~/.zshrc && open -e ~/.zshrc
```

Step 2 — append the function

Add the following block at the end of `~/.zshrc`, adjusting the two absolute paths to match your machine.

```
# flow-to-apex shortcut
convert-flow() {
  if [ -z "$1" ]; then
    echo "Usage: convert-flow <FlowApiName>"
    return 1
  fi
  node /Users/macbookair/Desktop/Bold/A_Projects/flow-to-apex/src/cli.js --sfdx \
    --source /Users/macbookair/Desktop/Bold/A_Projects/flowToApex/force-app/main/default/flows/$1.f
    --output /Users/macbookair/Desktop/Bold/A_Projects/flowToApex/force-app/main/default
}
```

Step 3 — reload the shell

Save the file and run the following so the current terminal picks up the new function. New terminal windows will load it automatically.

```
source ~/.zshrc
```

Step 4 — verify

```
convert-flow
```

You should see `Usage: convert-flow <FlowApiName>`. If you see `command not found`, repeat Step 3.

5. Quick start

Once the shortcut is set up, converting a Flow looks like this:

```
convert-flow Lead_Set_Status_On_Create
```

Expected output

```
Parsed Flow: Lead Set Status On Create
Object: Lead
Trigger: RecordBeforeSave (Create)
Elements: 2
wrote force-app/main/default/triggers/LeadTrigger.trigger
wrote force-app/main/default/triggers/LeadTrigger.trigger-meta.xml
wrote force-app/main/default/classes/LeadSetStatusOnCreateHandler.cls
wrote force-app/main/default/classes/LeadSetStatusOnCreateHandler.cls-meta.xml
```

```
Done. 4 files written to force-app/main/default (SFDX layout)
```

Four files appear inside your Salesforce DX project, automatically placed in `triggers/` and `classes/` subdirectories so they can be deployed without any further reorganisation.

6. Full migration workflow

Running the converter is one step in a larger Flow-to-Apex migration. The full end-to-end workflow looks like this:

- **1. Source the Flow** — either retrieve it from the org (SFDX: Retrieve Source in Manifest from Org) or place the `.flow-meta.xml` file directly into `force-app/main/default/flows/`.
- **2. Run the converter** — `convert-flow <FlowApiName>`.
- **3. Review the generated Apex** — open the trigger and handler in VS Code, verify the logic matches the original Flow, and look for any `// TODO` or `unsupported-element` warnings in the header.
- **4. Write a matching Apex test class** — the converter does not generate tests automatically (planned for Phase 2). At minimum, cover the positive case, the negative case, and a 200-record bulk scenario.
- **5. Validate against the org** — right-click `force-app/main/default` in VS Code and choose *SFDX: Deploy Source to Org* (or `sf project deploy validate` on the command line) with `RunLocalTests`.
- **6. Deactivate the source Flow** — in Setup → Flows, deactivate the Flow you just converted so it does not fire alongside the new trigger.
- **7. Deploy** — deploy the new trigger, handler, test class, and deactivated Flow to the target org in a single release.
- **8. Verify behaviour** — create or update a record that would have triggered the original Flow and confirm the Apex trigger now performs the same work. Use Debug Logs for added confidence.

7. Understanding the output

The trigger file

Every generated trigger is a one-liner that delegates to the handler. This keeps the trigger file free of logic and makes it easy to compose multiple handlers per object later.

```
trigger LeadTrigger on Lead (before insert) {
    LeadsetStatusOnCreateHandler.beforeSave(Trigger.new);
}
```

The handler class

The handler contains the converted logic, wrapped in a for loop over records (or, for after-save Flows, with a per-fire `Map<Id, SObject> recordsToUpdate` collector outside the per-record loop and a single bulk update at the end).

A header comment block records the source Flow's label, trigger type, and any Flow elements that were skipped because they fall outside the converter's current scope. **Always read the header.**

Bulkification example

Where the source Flow loops through records and does a DML inside the loop body, the converter detects the antipattern and rewrites it. For example, a Flow that loops through related Contacts and updates each one becomes:

```
// Loop: Loop Contacts
// Bulkification: DML inside loop detected (Update_Contact).
// DML lifted out of loop into Map<Id, Contact>.
Map<Id, Contact> loopContactsToUpdate = new Map<Id, Contact>();
for (Contact contactsItem : getRelatedContacts) {
    contactsItem.Phone = record.Phone;
    loopContactsToUpdate.put(contactsItem.Id, contactsItem);
}
if (!loopContactsToUpdate.isEmpty()) {
    update loopContactsToUpdate.values();
}
```

This is the central correctness concern of the converter — naive 1:1 mapping would put the update inside the inner loop and the code would fail governor limits at scale.

8. Supported & unsupported elements

Supported (this version)

Area	Notes
Flow type	Record-Triggered only (RecordBeforeSave / RecordAfterSave)
Record operations	Create, Update, Create & Update
recordLookups	Get Records — emitted as SOQL with WHERE clause
recordUpdates	Update Records — bulkified inside loops
recordCreates	Create Records — insert DML
recordDeletes	Delete Records — delete DML
decisions	Decision — compiled into if / else if / else
loops	Loop — compiled into for with bulkification
assignments	Assignment — field writes and collection adds
Start entry filter	Compiled into a continue guard inside the per-record loop

Not supported — parked for later phases

Element	Why it's parked
screens	Screen Flows need a UI layer (LWC) — out of scope for code-only conversion
subflows	Recursive subflow expansion needs cross-Flow analysis; planned for Phase 2
actionCalls	Invocable Apex / Platform Event / Callout actions have heterogeneous semantics
waits	Async / Pause requires Schedulable + Platform Event re-entry
recordRollbacks	Custom transaction control; rare in practice
Custom Apex Plugin Calls	Out-of-org plugin invocation; not in MVP scope
Fault paths	Flow's fault semantics don't map cleanly to Apex try/catch

When the parser encounters one of the parked element types, the generator emits a warning comment block at the top of the handler class listing what was skipped, so reviewers know the conversion is incomplete.

9. What you still have to do by hand

The output is a high-quality starting point, not deployable production code. Before pushing to production, review and add:

- **CRUD / FLS checks** — e.g. wrap SOQL with `WITH SECURITY_ENFORCED` or `Security.stripInaccessible()`.
- **Field-aware SOQL** — the converter currently emits `SELECT Id` and relies on dirty-field tracking. Add explicit field projections if you read multiple fields.
- **Apex tests** — positive, negative, and 200-record bulk cases at a minimum. Aim for > 75% coverage on the handler.
- **Recursion guard** — if other triggers or handlers also fire on the same object, add a static flag or a trigger-framework guard.
- **Formula translation** — Flow formulas don't have a direct Apex equivalent. If your Flow uses formulas, port them manually.
- **Operator coverage** — common operators (Equals, NotEqualTo, comparisons, IsNull, Contains) are mapped. Anything else lands as a `// TODO` in the generated code; review and finish them.

10. Troubleshooting

"command not found: convert-flow"

The shell function isn't loaded in the current terminal. Run `source ~/.zshrc` to load it, or open a new terminal. If new terminals still don't pick it up, confirm the function block was saved at the end of `~/.zshrc` and that the file has no syntax errors.

"Parse error: source file not found"

The Flow API name you passed does not match a file inside `force-app/main/default/flows/`. Open that folder in VS Code and check the exact file name. The argument to `convert-flow` should be the file name *without* the `.flow-meta.xml` suffix and with underscores instead of spaces.

"Unsupported Flow type"

The Flow is not a Record-Triggered Flow (e.g. it is a Screen Flow, Autolaunched Flow without record trigger, Scheduled Flow, or Platform Event-Triggered Flow). These are intentionally rejected in this version. Use a Record-Triggered Flow or wait for a future phase.

Deploy fails with "governor limits exceeded" in tests

Your Apex test class is probably testing at small scale. The generated handler is bulk-safe; the failure is almost always inside test data setup that does its own DML inside loops. Refactor the test data factory to insert lists in a single DML call.

Generated code references a field that doesn't exist

The source Flow references a field that exists in one org but not another (often a custom field). Check whether the field is missing in the target org and create or deploy it before retrying.

11. Phase 2 candidates

If this POC is approved for follow-up work, these are the natural next investments in priority order:

- **CRUD / FLS injection** — one config flag, large credibility win for security review.
- **Apex test class generator** — emit a baseline test alongside the handler with the right positive, negative, and bulk skeletons.
- **Field-aware SOQL** — track which fields the converted logic reads or writes and select them explicitly.
- **Subflow and Action Call support** — chain-expand subflows; translate Invocable Apex action calls into direct method calls.
- **Screen Flow logic-only extraction** — leave the UI in Flow / LWC and move only the business logic into Apex.
- **Direct deployment** — wrap the converter with a `sf project deploy start` call so a single command converts and deploys.
- **Apex → Flow direction** — only if there's a strong business reason; expect 30–50% pattern coverage at best.

Appendix — CLI reference

If you need to call the converter directly (without the `convert-flow` shortcut), the full CLI looks like this:

```
node <path-to-tool>/src/cli.js \
  --source <path-to-flow-xml> \
  --output <path-to-output-dir> \
  [--sfdx]
```

Flags

Flag	Purpose
<code>--source, -s</code>	Path to the <code>.flow-meta.xml</code> input file. Required.
<code>--output, -o</code>	Directory to write the generated files into. Required.
<code>--sfdx</code>	Use Salesforce DX layout: files go into <code>triggers/</code> and <code>classes/</code> subdirectories automatically.
<code>--help, -h</code>	Show built-in help and exit.